

Momentum Trader Codebase

src/config.py

```
from dataclasses import dataclass

@dataclass
class Config:
    # Layer 4 - Position Sizing & Risk Control
    CAPITAL: float = 500000.0      # Total Portfolio Capital (â‚¹)
    RISK_PER_TRADE: float = 0.01    # 1% risk rule
    MAX_HEAT: float = 0.05          # 5% max portfolio heat (max 5 open positions)

    # Layer 2 - Stock Universe & Screening Filters
    MIN_PRICE: float = 50.0         # Avoid tick slippage on penny stocks
    MIN_TURNOVER: float = 50000000.0 # â‚¹15 crore average daily turnover
    MIN_ATR_PCT: float = 1.5        # Volatility filter (min 1.5% daily move)
    MAX_ATR_PCT: float = 5.0        # Volatility filter (max 5% daily move)

    # Technicals
    DAYS_BACK: int = 900            # Ensure enough history for 200DMA and 52W Highs
```

src/data_extractor.py

```
import yfinance as yf
from datetime import datetime, timedelta
import pandas as pd
import logging

logger = logging.getLogger(__name__)

class DataFetcher:
    def __init__(self, days_back: int = 900, as_of_date: str = None):
        self.days_back = days_back
        self.as_of_date = as_of_date
        self.cached_data = None
        self.cached_tickers = []

    def get_start_end_dates(self):
        if self.as_of_date:
            end_date_obj = datetime.strptime(self.as_of_date, '%Y-%m-%d')
        else:
            end_date_obj = datetime.today()

        start_date = (end_date_obj - timedelta(days=self.days_back)).strftime('%Y-%m-%d')
        # yfinance end_date is exclusive, so we add 1 day to ensure the as_of_date is included
        end_date = (end_date_obj + timedelta(days=1)).strftime('%Y-%m-%d')
        return start_date, end_date

    def preload_data(self, tickers: list[str]):
        start_date, end_date = self.get_start_end_dates()
        try:
            logger.info(f"Bulk downloading data for {len(tickers)} tickers...")
            data = yf.download(tickers, start=start_date, end=end_date, progress=False)
            self.cached_data = data
            self.cached_tickers = tickers
        except Exception as e:
            logger.error(f"Error in bulk download: {e}")
            self.cached_data = None
            self.cached_tickers = []

    def fetch_historical_data(self, ticker: str, start_date: str, end_date: str) -> pd.DataFrame:
        if self.cached_data is not None and ticker in self.cached_tickers:
            if len(self.cached_tickers) > 1:
                try:
                    df = self.cached_data.xs(ticker, axis=1, level=1).dropna(how='all')
                except KeyError:
                    df = pd.DataFrame()
            else:
                df = self.cached_data.dropna(how='all')

            if not df.empty:
                # timezone naive vs aware index fix
                if df.index.tz is not None:
                    df.index = df.index.tz_localize(None)
                # Slice data between start_date and end_date (exclusive of end_date as per yfinance
                behavior)
                sliced_df = df.loc[(df.index >= pd.to_datetime(start_date)) & (df.index <
```

```

pd.to_datetime(end_date))
    if not sliced_df.empty:
        return sliced_df
    else:
        return pd.DataFrame()

    try:
        data = yf.Ticker(ticker).history(start=start_date, end=end_date)
        if data.empty:
            logger.warning(f"No data found for {ticker} between {start_date} and {end_date}.")
            return pd.DataFrame()
        return data
    except Exception as e:
        logger.error(f"Error fetching data for {ticker}: {e}")
        return pd.DataFrame()

def fetch_multiple_stocks(self, tickers: list[str]) -> dict[str, pd.DataFrame]:
    start_date, end_date = self.get_start_end_dates()
    stock_data = {}
    for ticker in tickers:
        df = self.fetch_historical_data(ticker, start_date, end_date)
        if not df.empty:
            stock_data[ticker] = df
    return stock_data

def fetch_market_indices(self) -> dict[str, pd.DataFrame]:
    """Fetches Nifty 50 and India VIX for Market Regime filters."""
    start_date, end_date = self.get_start_end_dates()
    return {
        "NIFTY_50": self.fetch_historical_data("^NSEI", start_date, end_date),
        "INDIA_VIX": self.fetch_historical_data("^INDIAVIX", start_date, end_date)
    }

```

src/indicators.py

```

import pandas as pd
import numpy as np

def calculate_atr(df, period=14):
    high_low = df['High'] - df['Low']
    high_close = np.abs(df['High'] - df['Close'].shift())
    low_close = np.abs(df['Low'] - df['Close'].shift())
    tr = pd.concat([high_low, high_close, low_close], axis=1).max(axis=1)
    return tr.rolling(window=period).mean()

def calculate_adx(df, period=14):
    plus_dm = df['High'].diff()
    minus_dm = df['Low'].diff()

    plus_dm[plus_dm < 0] = 0
    plus_dm[plus_dm < minus_dm] = 0
    minus_dm[minus_dm > 0] = 0
    minus_dm = minus_dm.abs()

    tr = calculate_atr(df, 1)
    atr_smooth = tr.ewm(alpha=1/period, adjust=False).mean()
    plus_di = 100 * (plus_dm.ewm(alpha=1/period, adjust=False).mean() / atr_smooth)
    minus_di = 100 * (minus_dm.ewm(alpha=1/period, adjust=False).mean() / atr_smooth)

    dx = (np.abs(plus_di - minus_di) / (plus_di + minus_di)) * 100
    adx = dx.ewm(alpha=1/period, adjust=False).mean()
    return adx

def calculate_rsi(df, period=2):
    delta = df['Close'].diff()
    gain = delta.mask(delta < 0, 0.0)
    loss = -delta.mask(delta > 0, 0.0)

    avg_gain = gain.ewm(alpha=1/period, adjust=False).mean()
    avg_loss = loss.ewm(alpha=1/period, adjust=False).mean()

    rs = avg_gain / avg_loss
    return 100 - (100 / (1 + rs))

def zscore(series):
    mean = series.mean()
    std = series.std(ddof=0)
    if std == 0 or np.isnan(std):
        return pd.Series(np.nan, index=series.index)
    return (series - mean) / std

def calculate_momentum_score(df):
    if len(df) < 252:

```

```

        return np.nan, np.nan

    ret_6m = df['Close'].iloc[-1] / df['Close'].iloc[-126] - 1
    vol_6m = df['Close'].pct_change().iloc[-126:].std()
    ret_12m = df['Close'].iloc[-1] / df['Close'].iloc[-252] - 1
    vol_12m = df['Close'].pct_change().iloc[-252:].std()

    if vol_6m <= 0 or vol_12m <= 0:
        return np.nan, np.nan

    return (ret_6m / vol_6m), (ret_12m / vol_12m)

def check_52w_breakout(df):
    if len(df) < 252:
        return False, {}

    current_price = df['Close'].iloc[-1]
    prev_52w_high = df['Close'].iloc[-253:-1].max()
    avg_vol_50 = df['Volume'].iloc[-51:-1].mean()
    ma50 = df['Close'].rolling(50).mean().iloc[-1]
    ma200 = df['Close'].rolling(200).mean().iloc[-1]
    ma50_prev = df['Close'].rolling(50).mean().iloc[-6]
    ma200_prev = df['Close'].rolling(200).mean().iloc[-6]

    breakout = current_price > prev_52w_high and current_price > df['Close'].iloc[-2]
    volume_surge = avg_vol_50 > 0 and df['Volume'].iloc[-1] >= 1.5 * avg_vol_50
    trend_ok = current_price > ma50 and current_price > ma200
    dma_rising = ma50 > ma50_prev and ma200 > ma200_prev

    # 3-week base check: past 15 days max/min shouldn't vary by more than 15%
    base_high = df['High'].iloc[-16:-1].max()
    base_low = df['Low'].iloc[-16:-1].min() # Actually we want min
    base_low = df['Low'].iloc[-16:-1].min()
    base_tight = False
    if base_low > 0:
        base_tight = (base_high / base_low - 1) < 0.15

    is_candidate = breakout and volume_surge and trend_ok and dma_rising and base_tight

    # Stop loss: low of breakout candle or recent swing low (using 5 day low)
    swing_low = df['Low'].iloc[-5:].min()
    candle_low = df['Low'].iloc[-1]
    stop_loss = min(swing_low, candle_low)

    return is_candidate, {"stop_loss": stop_loss}

def check_20dma_pullback(df):
    if len(df) < 200:
        return False, {}

    current_price = df['Close'].iloc[-1]
    ma20 = df['Close'].rolling(20).mean().iloc[-1]
    ma50 = df['Close'].rolling(50).mean().iloc[-1]
    ma200 = df['Close'].rolling(200).mean().iloc[-1]
    rsi2 = calculate_rsi(df, 2).iloc[-1]

    near_20dma = abs(current_price - ma20) / ma20 <= 0.01
    in_uptrend = current_price > ma50 and current_price > ma200
    bullish_reversal = df['Close'].iloc[-1] > df['Open'].iloc[-1] and df['Close'].iloc[-1] >
df['Close'].iloc[-2]

    is_candidate = in_uptrend and near_20dma and rsi2 < 10 and bullish_reversal

    # Stop loss: low of signal candle or swing low
    swing_low = df['Low'].iloc[-5:].min()
    candle_low = df['Low'].iloc[-1]
    stop_loss = min(swing_low, candle_low)

    # Target 1: prior swing high
    swing_high = df['High'].iloc[-20:-1].max()

    return is_candidate, {"stop_loss": stop_loss, "swing_high": swing_high}

```

src/screening.py

```

from src.indicators import (
    calculate_atr, calculate_adx, calculate_rsi,
    calculate_momentum_score, check_52w_breakout, check_20dma_pullback
)
from datetime import datetime
import yfinance as yf
import pandas as pd

```

```

import logging

logger = logging.getLogger(__name__)

class StockScreener:
    def __init__(self, data_fetcher, config):
        self.data_fetcher = data_fetcher
        self.config = config

    def check_market_regime(self):
        """Layer 1: Market Regime Check"""
        indices = self.data_fetcher.fetch_market_indices()
        nifty = indices.get("NIFTY 50")
        vix = indices.get("INDIA_VIX")

        regime = {
            "all longs off": False,
            "reduce_size_50_nifty": False,
            "vix_below_15": False,
            "reduce_size_25_vix": False,
            "reduce_size_50_vix": False,
            "stop_widen_atr": 0.0,
            "no_breakouts": False,
            "trend_preferred": False,
            "mean_reversion_preferred": False
        }

        if nifty is not None and not nifty.empty and len(nifty) >= 200:
            current_nifty = nifty['Close'].iloc[-1]
            ma50 = nifty['Close'].rolling(50).mean().iloc[-1]
            ma200 = nifty['Close'].rolling(200).mean().iloc[-1]
            if current_nifty < ma200:
                regime["all longs off"] = True
            if current_nifty < ma50:
                regime["reduce_size_50_nifty"] = True

            adx = calculate_adx(nifty, 14).iloc[-1]
            if adx > 25:
                regime["trend_preferred"] = True
            elif adx < 20:
                regime["mean_reversion_preferred"] = True

        if vix is not None and not vix.empty:
            current_vix = vix['Close'].iloc[-1]
            if current_vix < 15:
                regime["vix_below_15"] = True
            elif 15 <= current_vix <= 20:
                regime["reduce_size_25_vix"] = True
                regime["stop_widen_atr"] = 1.0 # Widen stops by 1 ATR
            elif current_vix > 20:
                regime["reduce_size_50_vix"] = True
                regime["no_breakouts"] = True

        return regime

    def calculate_average_turnover(self, df, period=20):
        df['Turnover'] = df['Volume'] * df['Close']
        return df['Turnover'].rolling(period).mean().iloc[-1]

    def screen_stocks(self, tickers):
        regime = self.check_market_regime()
        if regime.get("all longs off", False):
            return pd.DataFrame(), regime

        start_date, end_date = self.data_fetcher.get_start_end_dates()
        passed_stocks = []

        for ticker in tickers:
            try:
                df = self.data_fetcher.fetch_historical_data(ticker, start_date, end_date)
                if df.empty or len(df) < 252:
                    continue

                current_price = df['Close'].iloc[-1]
                if current_price <= self.config.MIN_PRICE:
                    continue

                avg_turnover = self.calculate_average_turnover(df)
                if avg_turnover < self.config.MIN_TURNOVER:
                    continue

                atr_series = calculate_atr(df, 14)
                atr = atr_series.iloc[-1]

```

```

atr_pct = (atr / current_price) * 100
if not (self.config.MIN_ATR_PCT <= atr_pct <= self.config.MAX_ATR_PCT):
    continue

avg_vol_50 = df['Volume'].iloc[-51:-1].mean()
vol_surge = df['Volume'].iloc[-1] / avg_vol_50 if avg_vol_50 > 0 else 1.0

m6, m12 = calculate_momentum_score(df)
is_breakout, breakout_info = check_52w_breakout(df)
is_pullback, pullback_info = check_20dma_pullback(df)

passed_stocks.append({
    'Ticker': ticker,
    'Price': current_price,
    'Average_Turnover': avg_turnover,
    'ATR': atr,
    'ATR_%': atr_pct,
    'Vol_Surge': vol_surge,
    'M6': m6,
    'M12': m12,
    'Breakout_Candidate': is_breakout,
    'Breakout_Info': breakout_info,
    'Pullback_Candidate': is_pullback,
    'Pullback_Info': pullback_info,
    'RSI2': calculate_rsi(df, 2).iloc[-1]
})

except Exception as e:
    logger.error(f"Error screening {ticker}: {e}")
    continue

df_result = pd.DataFrame(passed_stocks) if passed_stocks else pd.DataFrame()
if not df_result.empty:
    from src.indicators import zscore
    df_result['M6_Z'] = zscore(df_result['M6'])
    df_result['M12_Z'] = zscore(df_result['M12'])
    df_result['Momentum_Score'] = (df_result['M6_Z'] + df_result['M12_Z']) / 2
    df_result = df_result.sort_values(by='Momentum_Score', ascending=False)

return df_result, regime

```

src/simulator.py

```

import yfinance as yf
import pandas as pd
from datetime import datetime, timedelta
import logging

logger = logging.getLogger(__name__)

def simulate_trades(trade_plans_data, as_of_date, data_fetcher=None):
    if not trade_plans_data:
        return pd.DataFrame()

    start_date = (datetime.strptime(as_of_date, '%Y-%m-%d') + timedelta(days=1)).strftime('%Y-%m-%d')
    end_date = datetime.today().strftime('%Y-%m-%d')

    results = []

    for plan in trade_plans_data:
        ticker = plan['Ticker']
        entry_price = plan['Entry_Price']
        qty = plan['Quantity']
        t1 = plan['T1']
        t2 = plan['T2']
        initial_stop = plan['Stop_Loss']

        logger.info(f"Simulating {ticker} from {start_date}...")

        try:
            if data_fetcher:
                df = data_fetcher.fetch_historical_data(ticker, start_date, end_date)
            else:
                import yfinance as yf
                df = yf.Ticker(ticker).history(start=start_date, end=end_date)

            if df.empty:
                logger.warning(f"No forward data for {ticker}")
                continue

            status = 'Open'
            exit_price = None

```

```

        exit_date = None
        current_stop = initial_stop
        for date, row in df.iterrows():
            low = row['Low']
            high = row['High']

            # Check for stop loss hit
            if low <= current_stop:
                # If gap down below stop loss, exit at Open
                exit_price = row['Open'] if row['Open'] < current_stop else current_stop
                status = 'Closed (Stop Loss)' if current_stop == initial_stop else 'Closed (Trailing
Stop)'

                exit_date = date.strftime('%Y-%m-%d')
                break

            # Check for T2 hit
            if high >= t2:
                # Exit at T2
                exit_price = row['Open'] if row['Open'] > t2 else t2
                status = 'Closed (T2 Hit)'
                exit_date = date.strftime('%Y-%m-%d')
                break

            # Check for T1 hit (trail stop to entry)
            if high >= t1 and current_stop < entry_price:
                current_stop = entry_price

        # If still open, use last close
        if status == 'Open':
            exit_price = df['Close'].iloc[-1]
            exit_date = df.index[-1].strftime('%Y-%m-%d')

        profit = (exit_price - entry_price) * qty
        profit_pct = (exit_price - entry_price) / entry_price * 100

        results.append({
            'Ticker': ticker,
            'Strategy': plan['Strategy'],
            'Entry Date': as_of_date,
            'Entry Price': round(entry_price, 2),
            'Quantity': qty,
            'Status': status,
            'Exit Date': exit_date,
            'Exit Price': round(exit_price, 2),
            'Profit/Loss (â‚¬)': round(profit, 2),
            'Return (%)': round(profit_pct, 2)
        })

    except Exception as e:
        logger.error(f"Error simulating {ticker}: {e}")

    return pd.DataFrame(results)

```

src/trade_plan.py

```

import pandas as pd
import numpy as np

class TradePlanner:
    def __init__(self, config, regime):
        self.config = config
        self.regime = regime

    def display_trade_plan(self, row, label, risk_amount_rs, stop_widen_atr):
        entry_price = row['Price']
        atr = row['ATR']

        stop_loss = None
        t1_target = entry_price + 1.5 * atr
        t2_target = entry_price + 3 * atr

        if label == "Breakout":
            stop_loss = row.get('Breakout_Info', {}).get('stop_loss')
        elif label == "Pullback":
            stop_loss = row.get('Pullback_Info', {}).get('stop_loss')
            swing_high = row.get('Pullback_Info', {}).get('swing_high')
            if swing_high and not pd.isna(swing_high) and swing_high > entry_price:
                t1_target = swing_high

        if stop_loss is None or pd.isna(stop_loss):
            stop_loss = entry_price - atr * (1 + stop_widen_atr)
        else:

```

```

        if stop_widen_atr > 0:
            stop_loss = stop_loss - (atr * stop_widen_atr)

stop_distance_pct = ((entry_price - stop_loss) / entry_price) * 100

if stop_distance_pct > 8.0: # Allow up to 8% for midcaps as per document
    return f"{label} [{row['Ticker']}] SKIP: Stop loss too wide ({stop_distance_pct:.1f}%).",
None

risk_per_share = entry_price - stop_loss
if risk_per_share <= 0:
    return f"{label} [{row['Ticker']}] SKIP: Invalid stop level.", None

quantity = int(risk_amount_rs // risk_per_share)
if quantity < 1:
    return f"{label} [{row['Ticker']}] SKIP: Position size too small after risk sizing.", None

position_value = quantity * entry_price
trade_plan = (
    f"{label} [{row['Ticker']}] Price: â{entry_price:.2f} | ATR%: {row['ATR_%']:.2f}% | Vol
Surge: {row['Vol_Surge']:.1f}x\n"
    f"    â° Buy {quantity} shares | Cost: â,{position_value:.2f} | Risk:
â,{risk_amount_rs:.0f}\n"
    f"    â° Stop Loss: â,{stop_loss:.2f} | T1: â,{t1_target:.2f} | T2: â,{t2_target:.2f}\n"
    f"    â° Rule: If T1 hits, move stop to breakeven. Trail remaining at 10-EMA or prior swing
low.\n"
    f"    â° Notes: Check earnings/settings/SEBI news before entry."
)

plan_dict = {
    "Strategy": label,
    "Ticker": row['Ticker'],
    "Entry_Price": entry_price,
    "Quantity": quantity,
    "Cost": position_value,
    "Stop_Loss": stop_loss,
    "T1": t1_target,
    "T2": t2_target,
    "ATR": atr
}
return trade_plan, plan_dict

def generate_trade_plans(self, screener_df):
    risk_amount_rs = self.config.CAPITAL * self.config.RISK_PER_TRADE

    if self.regime.get("reduce_size_50_nifty") or self.regime.get("reduce_size_50_vix"):
        risk_amount_rs *= 0.5
    elif self.regime.get("reduce_size_25_vix"):
        risk_amount_rs *= 0.75

    stop_widen_atr = self.regime.get("stop_widen_atr", 0.0)

    momentum_candidates = screener_df.head(5)

    if 'Breakout_Candidate' in screener_df.columns:
        if self.regime.get("no_breakouts") or self.regime.get("mean_reversion_preferred"):
            breakout_candidates = pd.DataFrame()
        else:
            breakout_candidates =
screener_df[screener_df['Breakout_Candidate']].sort_values(by='Vol_Surge', ascending=False).head(3)
    else:
        breakout_candidates = pd.DataFrame()

    if 'Pullback_Candidate' in screener_df.columns and 'RSI2' in screener_df.columns:
        pullback_candidates =
screener_df[screener_df['Pullback_Candidate']].sort_values(by='RSI2').head(3)
    else:
        pullback_candidates = pd.DataFrame()

    trade_plans_str = []
    trade_plans_data = []

    def process_row(row, label):
        plan_str, plan_dict = self.display_trade_plan(row, label, risk_amount_rs, stop_widen_atr)
        trade_plans_str.append(plan_str)
        if plan_dict:
            trade_plans_data.append(plan_dict)

    for _, row in momentum_candidates.iterrows():
        process_row(row, "Momentum")

    for _, row in breakout_candidates.iterrows():
        process_row(row, "Breakout")

```

```

for _, row in pullback_candidates.iterrows():
    process_row(row, "Pullback")

return trade_plans_str, trade_plans_data

```

main.py

```

import logging
import sys
import json
import argparse
import pandas as pd
from datetime import datetime, timedelta
from src.config import Config
from src.data_extractor import DataFetcher
from src.screening import StockScreener
from src.trade_plan import TradePlanner

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")

def main():
    parser = argparse.ArgumentParser(description="Momentum Trader")
    parser.add_argument("--as-of-date", type=str, help="Backtest as of a specific date (YYYY-MM-DD)")
    parser.add_argument("--start-date", type=str, help="Backtest start date for a range (YYYY-MM-DD)")
    parser.add_argument("--end-date", type=str, help="Backtest end date for a range (YYYY-MM-DD)")
    args = parser.parse_args()

    # Determine dates to process
    dates_to_process = []
    if args.start_date and args.end_date:
        start_dt = datetime.strptime(args.start_date, '%Y-%m-%d')
        end_dt = datetime.strptime(args.end_date, '%Y-%m-%d')
        current_dt = start_dt
        while current_dt <= end_dt:
            # Skip weekends
            if current_dt.weekday() < 5:
                dates_to_process.append(current_dt.strftime('%Y-%m-%d'))
                current_dt += timedelta(days=1)
    elif args.as_of_date:
        dates_to_process = [args.as_of_date]
    else:
        dates_to_process = [datetime.today().strftime('%Y-%m-%d')]

    # 1. Initialize Configuration
    config = Config()
    logging.info("Initialized configuration.")

    # Fetch Nifty 150 (Midcap 150)
    tickers = [
        '3MINDIA.NS', 'AARTIIND.NS', 'BBTC.NS', 'BHARATFORG.NS', 'CHOLAHLDNG.NS',
        'CRISIL.NS', 'HAL.NS', 'IDFCFIRSTB.NS', 'JKCEMENT.NS', 'M&MFIN.NS',
        'SKFINDIA.NS', 'TATACHEM.NS', 'TVSMOTOR.NS', 'WHIRLPOOL.NS', 'JSWENERGY.NS',
        'RECLTD.NS', 'TATAPOWER.NS', 'GICRE.NS', 'TATACOMM.NS', 'ABFRL.NS',
        'ASHOKLEY.NS', 'BANKBARODA.NS', 'GUJGASLTD.NS', 'LALPATHLAB.NS', 'PAGEIND.NS',
        'SUNDRMFAST.NS', 'TTKPRESTIG.NS', 'VINATIORGA.NS', 'ABCAPITAL.NS',
        'ASTRAZEN.NS', 'BATAINDIA.NS', 'CANBK.NS', 'COROMANDEL.NS', 'CUMMINSIND.NS',
        'ENDURANCE.NS', 'GLAXO.NS', 'JMFINANCIL.NS', 'MPHASIS.NS', 'OFSS.NS',
        'PERSISTENT.NS', 'PFIZER.NS', 'PGHL.NS', 'SUNTV.NS', 'THERMAX.NS', 'APLLTD.NS',
        'BHEL.NS', 'DIXON.NS', 'GLENMARK.NS', 'JINDALSTEL.NS', 'OBEROIIRLT.NS',
        'PIIND.NS', 'SANOFI.NS', 'EMAMILTD.NS', 'ESCORTS.NS', 'FORTIS.NS',
        'HINDZINC.NS', 'INDHOTEL.NS', 'NHPC.NS', 'SCHAEFFLER.NS', 'SUPREMEIND.NS',
        'BANKINDIA.NS', 'CHOLAFIN.NS', 'EXIDEIND.NS', 'IDBI.NS', 'MANAPPURAM.NS',
        'METROPOLIS.NS', 'MOTILALOFS.NS', 'NATCOPHARM.NS', 'SOLARINDS.NS', 'ZEEL.NS',
        'AUBANK.NS', 'CASTROLIND.NS', 'DALBHARAT.NS', 'POLYCAB.NS', 'SAIL.NS',
        'TIINDIA.NS', 'VGUARD.NS', 'ASTRAL.NS', 'ATGL.NS', 'DEEPAKNT.NS',
        'LAURUSLABS.NS', 'LTTS.NS', 'NAVINFLUOR.NS', 'BAYERCROP.NS', 'CONCOR.NS',
        'GODREJPROP.NS', 'IDEA.NS', 'PFC.NS', 'PRESTIGE.NS', 'ABB.NS', 'APOLLOTYRE.NS',
        'BEL.NS', 'HATSUN.NS', 'NIACL.NS', 'RAJESHEXPO.NS', 'RELAXO.NS',
        'SUNDARMFIN.NS', 'TRENT.NS', 'UNIONBANK.NS', 'AJANTPHARM.NS', 'CESC.NS',
        'CROMPTON.NS', 'FEDERALBNK.NS', 'GODREJAGRO.NS', 'GODREJIND.NS', 'IPCALAB.NS',
        'KANSAINER.NS', 'RBLBANK.NS', 'SVJN.NS', 'SUMICHEM.NS', 'ATUL.NS',
        'CREDITACC.NS', 'INDIAMART.NS', 'IRCTC.NS', 'LICHSGFIN.NS', 'OIL.NS',
        'RAMCOCEM.NS', 'SYNGENE.NS', 'VOLTAS.NS', 'AAVAS.NS', 'AIAENG.NS',
        'COFORGE.NS', 'HONAUT.NS', 'MFSL.NS', 'NAM-INDIA.NS', 'PHOENIXLTD.NS',
        'SRF.NS', 'TATAELXSI.NS', 'TORNTPOWER.NS', 'BALKRISIND.NS', 'CUB.NS',
        'GILLETTE.NS', 'ITI.NS', 'MAXHEALTH.NS', 'MGL.NS', 'VBL.NS', 'ZYDUSWELL.NS'
    ]
    logging.info(f"Loaded {len(tickers)} hardcoded tickers from Nifty Midcap 150.")

    # Setup Data Fetcher
    # Use the earliest date to preload enough data for the whole range

```

```

earliest_date = dates_to_process[0]
data_fetcher = DataFetcher(days_back=config.DAYS_BACK, as_of_date=earliest_date)
# Actually, we need data up to the latest date. Modify DataFetcher internally or just override dates
here.
start_fetch = (datetime.strptime(earliest_date, '%Y-%m-%d') -
timedelta(days=config.DAYS_BACK)).strftime('%Y-%m-%d')
end_fetch = (datetime.strptime(dates_to_process[-1], '%Y-%m-%d') + timedelta(days=1)).strftime('%Y-
%m-%d')

# Temporarily monkey-patch get_start_end_dates for preload to fetch the FULL range
def custom_get_dates():
    return start_fetch, end_fetch
data_fetcher.get_start_end_dates = custom_get_dates

# Preload all ticker data at once to significantly improve performance
data_fetcher.preload_data(tickers)

# Restore original method behavior so it dynamically updates based on data_fetcher.as_of_date
del data_fetcher.get_start_end_dates

all_results_dfs = []

for process_date in dates_to_process:
    logging.info(f"\n{'='*60}\nProcessing for Date: {process_date}\n{'='*60}")
    data_fetcher.as_of_date = process_date

    # Setup Screener
    screener = StockScreener(
        data_fetcher=data_fetcher,
        config=config
    )

    logging.info(f"Screening {len(tickers)} tickers...")
    screener_df, regime = screener.screen_stocks(tickers)

    if len(dates_to_process) == 1:
        print("\n" + "="*50)
        print("LAYER 1: MARKET REGIME (5-Point Check)")
        print("="*50)
        print(json.dumps(regime, indent=4))
        print("="*50 + "\n")

    if regime.get("all longs off", False):
        logging.info(f"Market is below 200-DMA on {process_date}. NO TRADES.")
        continue

    if screener_df.empty:
        logging.info(f"No stocks passed the screener on {process_date}.")
        continue

    logging.info(f"Found {len(screener_df)} candidates. Generating trade plans...")

    # Setup Trade Planner
    planner = TradePlanner(
        config=config,
        regime=regime
    )

    trade_plans_str, trade_plans_data = planner.generate_trade_plans(screener_df)

    if len(dates_to_process) == 1:
        print("\n" + "="*50)
        print("MOMENTUM TRADER - 5-LAYER TRADE PLANS")
        print("="*50 + "\n")
        for plan in trade_plans_str:
            print(plan)
            print("-" * 50)

    if trade_plans_data:
        from src.simulator import simulate_trades
        results_df = simulate_trades(trade_plans_data, process_date, data_fetcher)
        if not results_df.empty:
            all_results_dfs.append(results_df)

if all_results_dfs:
    final_results = pd.concat(all_results_dfs, ignore_index=True)
    # Assuming we save to excel or display
    if len(dates_to_process) > 1:
        excel_filename = f"backtest_results_{dates_to_process[0]}_to_{dates_to_process[-1]}.xlsx"
    else:
        excel_filename = f"backtest_results_{dates_to_process[0]}.xlsx"

    final_results.to_excel(excel_filename, index=False)

```

```

        logging.info(f"Simulation complete. Results saved to {excel_filename}")

        print("\n" + "="*50)
        print("SIMULATION RESULTS (PROFIT/LOSS)")
        print("="*50 + "\n")
        print(final_results[['Ticker', 'Strategy', 'Entry Date', 'Status', 'Return (%)', 'Profit/Loss
(â,¹)']].to_string(index=False))

        total_profit = final_results['Profit/Loss (â,¹)'].sum()
        print("\n" + "-"*50)
        print(f"TOTAL PROFIT/LOSS: â,¹{total_profit:,.2f}")
        print("-"*50)
    else:
        logging.info("No trades were simulated across the selected dates.")

if __name__ == "__main__":
    main()

```

fast_backtest.py

```

import argparse
import logging
import pandas as pd
import numpy as np
import yfinance as yf
from datetime import datetime, timedelta
import warnings

# Ignore pandas future warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

from src.config import Config

logging.basicConfig(level=logging.INFO, format="%asctime)s [%levelname)s] %(message)s")

def calculate_rsi_series(close, period=2):
    delta = close.diff()
    gain = delta.mask(delta < 0, 0.0)
    loss = -delta.mask(delta > 0, 0.0)
    avg_gain = gain.ewm(alpha=1/period, adjust=False).mean()
    avg_loss = loss.ewm(alpha=1/period, adjust=False).mean()
    rs = avg_gain / avg_loss
    return 100 - (100 / (1 + rs))

def calculate_atr_series(df, period=14):
    high_low = df['High'] - df['Low']
    high_close = np.abs(df['High'] - df['Close'].shift())
    low_close = np.abs(df['Low'] - df['Close'].shift())
    tr = pd.concat([high_low, high_close, low_close], axis=1).max(axis=1)
    return tr.rolling(window=period).mean()

def compute_signals(df, config):
    res = pd.DataFrame(index=df.index)
    res['Close'] = df['Close']
    res['Open'] = df['Open']
    res['High'] = df['High']
    res['Low'] = df['Low']
    res['Volume'] = df['Volume']

    res['Valid'] = (df['Close'] > config.MIN_PRICE)

    avg_turnover = (df['Volume'] * df['Close']).rolling(20).mean()
    res['Avg_Turnover'] = avg_turnover
    res['Valid'] = res['Valid'] & (avg_turnover >= config.MIN_TURNOVER)

    atr = calculate_atr_series(df, 14)
    atr_pct = (atr / df['Close']) * 100
    res['ATR'] = atr
    res['ATR_%'] = atr_pct
    res['Valid'] = res['Valid'] & (atr_pct >= config.MIN_ATR_PCT) & (atr_pct <= config.MAX_ATR_PCT)

    ret_6m = df['Close'] / df['Close'].shift(126) - 1
    vol_6m = df['Close'].pct_change().rolling(126).std()
    ret_12m = df['Close'] / df['Close'].shift(252) - 1
    vol_12m = df['Close'].pct_change().rolling(252).std()
    res['Momentum_Score'] = 0.5 * (ret_6m / vol_6m) + 0.5 * (ret_12m / vol_12m)

    prev_52w_high = df['Close'].shift(1).rolling(252).max()
    avg_vol_50 = df['Volume'].shift(1).rolling(50).mean()
    ma50 = df['Close'].rolling(50).mean()
    ma200 = df['Close'].rolling(200).mean()
    ma50_prev = ma50.shift(5)

```

```

ma200_prev = ma200.shift(5)

breakout = (df['Close'] > prev_52w_high) & (df['Close'] > df['Close'].shift(1))
volume_surge = (df['Volume'] >= 1.5 * avg_vol_50) & (avg_vol_50 > 0)
trend_ok = (df['Close'] > ma50) & (df['Close'] > ma200)
dma_rising = (ma50 > ma50_prev) & (ma200 > ma200_prev)
res['Breakout_Candidate'] = breakout & volume_surge & trend_ok & dma_rising

ma20 = df['Close'].rolling(20).mean()
rsi2 = calculate_rsi_series(df['Close'], 2)
near_20dma = (np.abs(df['Close'] - ma20) / ma20) <= 0.01
in_uptrend = (df['Close'] > ma50) & (df['Close'] > ma200)
bullish_reversal = (df['Close'] > df['Open']) & (df['Close'] > df['Close'].shift(1))
res['Pullback_Candidate'] = in_uptrend & near_20dma & (rsi2 < 10) & bullish_reversal

res['Vol_Surge'] = df['Volume'] / avg_vol_50
res['RSI2'] = rsi2

return res

def simulate_trades_fast(df_ticker, entry_idx, entry_price, initial_stop, t1, t2):
    current_stop = initial_stop

    lows = df_ticker['Low'].values
    highs = df_ticker['High'].values
    opens = df_ticker['Open'].values
    closes = df_ticker['Close'].values
    dates = df_ticker.index

    for i in range(len(lows)):
        low = lows[i]
        high = highs[i]

        if low <= current_stop:
            exit_price = opens[i] if opens[i] < current_stop else current_stop
            status = 'Closed (Stop Loss)' if current_stop == initial_stop else 'Closed (Trailing Stop)'
            return exit_price, dates[i], status

        if high >= t2:
            exit_price = opens[i] if opens[i] > t2 else t2
            status = 'Closed (T2 Hit)'
            return exit_price, dates[i], status

        if high >= t1 and current_stop < entry_price:
            current_stop = entry_price

    if len(closes) > 0:
        return closes[-1], dates[-1], 'Open'
    else:
        return entry_price, None, 'No Forward Data'

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--start-date", type=str, required=True, help="YYYY-MM-DD")
    parser.add_argument("--end-date", type=str, required=True, help="YYYY-MM-DD")
    args = parser.parse_args()

    config = Config()
    start_dt = datetime.strptime(args.start_date, '%Y-%m-%d')
    end_dt = datetime.strptime(args.end_date, '%Y-%m-%d')

    fetch_start = (start_dt - timedelta(days=900)).strftime('%Y-%m-%d')
    fetch_end = (end_dt + timedelta(days=100)).strftime('%Y-%m-%d')

    tickers = [
        '3MINDIA.NS', 'AARTIIND.NS', 'BBTC.NS', 'BHARATFORG.NS', 'CHOLAHLDNG.NS',
        'CRISIL.NS', 'HAL.NS', 'IDFCFIRSTB.NS', 'JKCEMENT.NS', 'M&MFIN.NS',
        'SKFINDIA.NS', 'TATACHEM.NS', 'TVSMOTOR.NS', 'WHIRLPOOL.NS', 'JSWENERGY.NS',
        'RECLTD.NS', 'TATAPOWER.NS', 'GICRE.NS', 'TATACOMM.NS', 'ABFRL.NS',
        'ASHOKLEY.NS', 'BANKBARODA.NS', 'GUJGASLTD.NS', 'LALPATHLAB.NS', 'PAGEIND.NS',
        'SUNDRMFAST.NS', 'TTKPRESTIG.NS', 'VINATIORG.NS', 'ABCAPITAL.NS',
        'ASTRAZEN.NS', 'BATAINDIA.NS', 'CANBK.NS', 'COROMANDEL.NS', 'CUMMINSIND.NS',
        'ENDURANCE.NS', 'GLAXO.NS', 'JMFINANCIL.NS', 'MPHASIS.NS', 'OFSS.NS',
        'PERSISTENT.NS', 'PFIZER.NS', 'PGHL.NS', 'SUNTV.NS', 'THERMAX.NS', 'APLLTD.NS',
        'BHEL.NS', 'DIXON.NS', 'GLENMARK.NS', 'JINDALSTEL.NS', 'OBEROIRLTY.NS',
        'PIIND.NS', 'SANOFI.NS', 'EMAMILTD.NS', 'ESCORTS.NS', 'FORTIS.NS',
        'HINDZINC.NS', 'INDHOTEL.NS', 'NHPC.NS', 'SCHAEFFLER.NS', 'SUPREMEIND.NS',
        'BANKINDIA.NS', 'CHOLAFIN.NS', 'EXIDEIND.NS', 'IDBI.NS', 'MANAPPURAM.NS',
        'METROPOLIS.NS', 'MOTILALOFS.NS', 'NATCOPHARM.NS', 'SOLARINDS.NS', 'ZEEL.NS',
        'AUBANK.NS', 'CASTROLIND.NS', 'DALBHARAT.NS', 'POLYCAB.NS', 'SAIL.NS',
        'TIINDIA.NS', 'VGUARD.NS', 'ASTRAL.NS', 'ATGL.NS', 'DEEPAKNTR.NS',
        'LAURUSLABS.NS', 'LTTS.NS', 'NAVINFLOUR.NS', 'BAYERCROP.NS', 'CONCOR.NS',
        'GODREJPROP.NS', 'IDEA.NS', 'PFC.NS', 'PRESTIGE.NS', 'ABB.NS', 'APOLLOTYRE.NS',
    ]

```

```

        'BEL.NS', 'HATSUN.NS', 'NIACL.NS', 'RAJESHEXPO.NS', 'RELAXO.NS',
        'SUNDARFIN.NS', 'TRENT.NS', 'UNIONBANK.NS', 'AJANTPHARM.NS', 'CESC.NS',
        'CROMPTON.NS', 'FEDERALBNK.NS', 'GODREJAGRO.NS', 'GODREJIND.NS', 'IPCALAB.NS',
        'KANSAINER.NS', 'RBLBANK.NS', 'SJVN.NS', 'SUMICHEM.NS', 'ATUL.NS',
        'CREDITACC.NS', 'INDIAMART.NS', 'IRCTC.NS', 'LICHSGFIN.NS', 'OIL.NS',
        'RAMCOCEM.NS', 'SYNGENE.NS', 'VOLTAS.NS', 'AAVAS.NS', 'AIAENG.NS',
        'COFORGE.NS', 'HONAUT.NS', 'MFSL.NS', 'NAM-INDIA.NS', 'PHOENIXLTD.NS',
        'SRF.NS', 'TATAELXSI.NS', 'TORNTPOWER.NS', 'BALKRISIND.NS', 'CUB.NS',
        'GILLETTE.NS', 'ITI.NS', 'MAXHEALTH.NS', 'MGL.NS', 'VBL.NS', 'ZYDUSWELL.NS'
    ]
    all_tickers = tickers + ['^NSEI', '^INDIAVIX']

    logging.info(f"Downloading data from {fetch_start} to {fetch_end} for {len(all_tickers)}
tickers...")
    raw_data = yf.download(all_tickers, start=fetch_start, end=fetch_end, progress=False)

    if raw_data.empty:
        logging.error("Failed to download any data.")
        return

    nifty = raw_data.xs('^NSEI', axis=1, level=1).dropna(how='all') if '^NSEI' in
raw_data.columns.get_level_values(1) else pd.DataFrame()
    vix = raw_data.xs('^INDIAVIX', axis=1, level=1).dropna(how='all') if '^INDIAVIX' in
raw_data.columns.get_level_values(1) else pd.DataFrame()

    if not nifty.empty:
        nifty['MA200'] = nifty['Close'].rolling(200).mean()

    logging.info("Calculating technical indicators across all stocks...")
    stock_signals = {}
    for ticker in tickers:
        try:
            if ticker not in raw_data.columns.get_level_values(1):
                continue
            df_ticker = raw_data.xs(ticker, axis=1, level=1).dropna(how='all')
            if df_ticker.empty or len(df_ticker) < 252:
                continue
            if df_ticker.index.tz is not None:
                df_ticker.index = df_ticker.index.tz_localize(None)

            signals = compute_signals(df_ticker, config)
            stock_signals[ticker] = signals
        except Exception as e:
            continue

    date_range = pd.date_range(start=start_dt, end=end_dt, freq='B')

    results_by_date = []

    for process_date in date_range:
        date_str = process_date.strftime('%Y-%m-%d')
        all_longs_off = False
        stop_widen_atr = 0.0

        if not nifty.empty:
            nifty_slice = nifty.loc[:process_date]
            if not nifty_slice.empty:
                current_nifty = nifty_slice['Close'].iloc[-1]
                ma200 = nifty_slice['MA200'].iloc[-1]
                if current_nifty < ma200:
                    all_longs_off = True

        if not vix.empty:
            vix_slice = vix.loc[:process_date]
            if not vix_slice.empty:
                current_vix = vix_slice['Close'].iloc[-1]
                if current_vix > 18:
                    stop_widen_atr = 0.5

        if all_longs_off:
            results_by_date.append({
                'Invest_Date': date_str,
                'Capital': config.CAPITAL,
                'Profit': 0.0,
                'Return_%': 0.0,
                'Trades': 0,
                'Note': 'Market Below 200DMA'
            })
            continue

    candidates = []
    for ticker, signals in stock_signals.items():
        sig_slice = signals.loc[:process_date]

```

```

if sig_slice.empty:
    continue

if sig_slice.index[-1] != process_date:
    continue

row = sig_slice.iloc[-1]
if not row['Valid']:
    continue

if pd.isna(row['Momentum_Score']):
    continue

candidates.append({
    'Ticker': ticker,
    'Price': row['Close'],
    'ATR': row['ATR'],
    'ATR_%': row['ATR_%'],
    'Vol_Surge': row['Vol_Surge'],
    'Momentum_Score': row['Momentum_Score'],
    'Breakout_Candidate': row['Breakout_Candidate'],
    'Pullback_Candidate': row['Pullback_Candidate'],
    'RSI2': row['RSI2']
})

if not candidates:
    results_by_date.append({
        'Invest_Date': date_str,
        'Capital': config.CAPITAL,
        'Profit': 0.0,
        'Return_%': 0.0,
        'Trades': 0,
        'Note': 'No Candidates Passed'
    })
    continue

df_cand = pd.DataFrame(candidates).sort_values(by='Momentum_Score', ascending=False)

momentum_candidates = df_cand.head(5)
breakout_candidates = df_cand[df_cand['Breakout_Candidate']].sort_values(by='Vol_Surge',
ascending=False).head(3)
pullback_candidates = df_cand[df_cand['Pullback_Candidate']].sort_values(by='RSI2').head(3)

risk_amount_rs = config.CAPITAL * config.RISK_PER_TRADE
trade_plans = []

def add_plan(row, label):
    entry_price = row['Price']
    atr = row['ATR']
    stop_loss = entry_price - atr * (1 + stop_widen_atr)
    t1_target = entry_price + 1.5 * atr
    t2_target = entry_price + 3 * atr

    stop_distance_pct = ((entry_price - stop_loss) / entry_price) * 100
    if stop_distance_pct > 5.0: return
    risk_per_share = entry_price - stop_loss
    if risk_per_share <= 0: return

    quantity = int(risk_amount_rs // risk_per_share)
    if quantity < 1: return

    trade_plans.append({
        'Strategy': label,
        'Ticker': row['Ticker'],
        'Entry_Price': entry_price,
        'Quantity': quantity,
        'Stop_Loss': stop_loss,
        'T1': t1_target,
        'T2': t2_target
    })

for _, row in momentum_candidates.iterrows(): add_plan(row, "Momentum")
for _, row in breakout_candidates.iterrows(): add_plan(row, "Breakout")
for _, row in pullback_candidates.iterrows(): add_plan(row, "Pullback")

if not trade_plans:
    results_by_date.append({
        'Invest_Date': date_str,
        'Capital': config.CAPITAL,
        'Profit': 0.0,
        'Return_%': 0.0,
        'Trades': 0,
        'Note': 'No Valid Trade Plans'
    })

```

```

    })
    continue

    daily_profit = 0.0
    trades_count = 0
    for plan in trade_plans:
        ticker = plan['Ticker']
        df_ticker = stock_signals[ticker]
        forward_data = df_ticker.loc[process_date:]
        if len(forward_data) <= 1:
            continue

        forward_data = forward_data.iloc[1:]

        exit_price, exit_date, status = simulate_trades_fast(
            forward_data, process_date, plan['Entry_Price'], plan['Stop_Loss'], plan['T1'],
plan['T2'])

        profit = (exit_price - plan['Entry_Price']) * plan['Quantity']
        daily_profit += profit
        trades_count += 1

    return_pct = (daily_profit / config.CAPITAL) * 100

    results_by_date.append({
        'Invest Date': date_str,
        'Capital': config.CAPITAL,
        'Profit': round(daily_profit, 2),
        'Return %': round(return_pct, 2),
        'Trades': trades_count,
        'Note': ''
    })

final_df = pd.DataFrame(results_by_date)
print("\n" + "="*80)
print("BACKTEST COMPARISON: RETURN BY INVESTMENT DATE")
print("="*80)
print(final_df.to_string(index=False))

if len(final_df) >= 2:
    start_ret = final_df.iloc[0]['Return %']
    end_ret = final_df.iloc[-1]['Return %']
    print("\n" + "="*80)
    print(f"Comparing Strategy Start Dates:")
    print(f"Investing on {final_df.iloc[0]['Invest Date']}: {start_ret:.2f}% Return")
    print(f"Investing on {final_df.iloc[-1]['Invest Date']}: {end_ret:.2f}% Return")
    print(f"Difference: {end_ret - start_ret:.2f}%")
    print("="*80)

    final_df.to_csv("backtest_comparison.csv", index=False)
    logging.info("Saved results to backtest_comparison.csv")

if __name__ == "__main__":
    main()

```